Part 01 : Theoretical Questions

Q1 : What is an interface in C#? Why do we use interfaces instead of depending on concrete classes directly? Mention at least three benefits of using interfaces.

Q2 : Look at the following code and answer the questions below:

```
interface IEnglishSpeaker
{
    void Greet();
}

interface IArabicSpeaker
{
    void Greet();
}

class Translator : IEnglishSpeaker, IArabicSpeaker
{
    public void Greet()
    {
        Console.WriteLine("Hello / Ahlan");
    }
}
```

a) What is the problem with this design? Both interfaces have a method called Greet() — how does the class handle it currently?

b) How would you fix this so IEnglishSpeaker.Greet() says "Hello" and IArabicSpeaker.Greet() says "Ahlan"? What is this technique called?

c) After applying your fix, can you call Greet() directly on a Translator object (e.g. translator.Greet())? Why or why not? How do you call each version?

Q3 : Explain the difference between a shallow copy and a deep copy. When would you use each one? What is the risk of using a shallow copy when the object has reference-type fields?

Q4 : Look at the following code and determine the output. Explain why.

```
class Department { public string Name; }
class Employee
{
    public string Title;
    public Department Dept;
    public Employee ShallowCopy() => (Employee)this.MemberwiseClone();
}

var e1 = new Employee { Title = "Dev", Dept = new Department { Name = "IT" } };
var e2 = e1.ShallowCopy();
e2.Title = "QA";
e2.Dept.Name = "Testing";

Console.WriteLine($"{e1.Title} - {e1.Dept.Name}");
Console.WriteLine($"{e2.Title} - {e2.Dept.Name}");
```

Part 02 : Practical (Extending the Movie Ticket Booking System)

In the previous assignments, you built a Movie Ticket Booking System with inheritance, polymorphism, and a Cinema class. Now you will refactor and extend the system using interfaces and object copying.

User Story :

The cinema manager wants to add three new capabilities to the booking system:

1. Unified Printing — The manager noticed that different parts of the system print ticket info in different ways. He wants a standard contract that guarantees any printable object in the system (tickets, receipts, etc.) can print itself through a single common method. All ticket types (Standard, VIP, IMAX) should follow this contract, and each one should print its own specific details. The Cinema should also be able to print all its tickets using this contract. There should also be a utility method in BookingHelper that can accept an array of any printable objects and print them all — without knowing their actual types.

2. Booking & Cancellation — Right now, tickets are created but there is no way to track whether a ticket is actually booked or cancelled. The manager wants every ticket to support booking and cancellation operations. A ticket can only be booked once (trying to book an already-booked ticket should fail), and can only be cancelled if it is currently booked (trying to cancel a non-booked ticket should fail). The booking status should appear when the ticket is printed.

3. Ticket Cloning — Sometimes a customer wants to buy a second ticket with the exact same details as an existing one but for a different movie. The system

should be able to create a full independent copy of any ticket (especially VIP tickets). Changing anything on the copy must NOT affect the original ticket.

Requirements :

Use interfaces to implement each feature above. Think about which standard C# interfaces (like ICloneable) can help you, and which custom interfaces you need to create yourself.

Your solution must demonstrate the following concepts from this session:

- Defining and implementing custom interfaces
- A class implementing multiple interfaces
- Using an interface as a method parameter (interface polymorphism)
- Implementing ICloneable for deep copying
- Proving that the cloned object is fully independent from the original

In Main, demonstrate :

a. Create a Cinema and open it.

b. Create one of each ticket type with hardcoded data. Book all three and add them to the Cinema.

c. Print all tickets through the Cinema.

d. Clone a VIP ticket, change the clone's movie name, and print both to prove independence.

e. Cancel one ticket and reprint it to show the updated status.

f. Use the utility method to print an array of printable tickets.

g. Close the Cinema.

ROUTE

Expected Output (Example) :

```
=== Cinema Opened ===

--- All Tickets ---
[Ticket #1] Inception | Standard | Seat: A5 | Price: 80 | After Tax: 91.2 |
Booked: Yes
[Ticket #2] Avengers | VIP | Lounge: Yes | Fee: 50 | Price: 200 | After Tax: 228
| Booked: Yes
[Ticket #3] Dune | IMAX | 3D: Yes | Price: 130 | After Tax: 148.2 | Booked: Yes

--- Clone Test ---
Original : [Ticket #2] Avengers | VIP | Lounge: Yes | Fee: 50 | Price: 200 |
After Tax: 228 | Booked: Yes
Clone    : [Ticket #4] Interstellar | VIP | Lounge: Yes | Fee: 50 | Price: 200 |
After Tax: 228 | Booked: No

--- After Cancellation ---
[Ticket #1] Inception | Standard | Seat: A5 | Price: 80 | After Tax: 91.2 |
Booked: No

--- BookingHelper.PrintAll ---
[Ticket #1] Inception | Standard | Seat: A5 | Price: 80 | After Tax: 91.2 |
Booked: No
[Ticket #2] Avengers | VIP | Lounge: Yes | Fee: 50 | Price: 200 | After Tax: 228
| Booked: Yes
[Ticket #3] Dune | IMAX | 3D: Yes | Price: 130 | After Tax: 148.2 | Booked: Yes

=== Cinema Closed ===
```